# Data Structures & Algorithm


## Submitted To:

**Mr. Mujtaba Kamal Pasha**

## Submitted By:


| | |
|---|---|
| **2024-CE-25:** | **Asmad Soban** |
| **2024-CE-46:** | **Ubaid Ali** |


## University of Engineering and Technology

# Data Structures & Algorithm

## Table of Contents



## Introduction

Graphs are one of the most powerful data structures in computer science, used to model networks, maps, and relationships between entities. This project applies graph theory to a real-world problem — city traffic navigation. A Smart City Traffic Navigation System is built that represents city intersections as nodes and roads as directed weighted edges. The system is designed to simulate one-way road traffic, allow dynamic expansion of the city map, and find the most efficient route between any two points. Dijkstra's Algorithm, which is the foundation of modern GPS navigation systems like Google Maps, is implemented at the core of this project.

## Problem Statement

In a smart city, vehicles need to navigate from one intersection to another through a network of roads. Some roads are one-way, just like real city traffic. This project simulates a city traffic system where intersections are nodes and roads are directed weighted edges. The system allows users to add new intersections and roads dynamically, find the shortest route between any two intersections using Dijkstra's Algorithm, and display the full city map. All data is loaded from and saved to text files, making the system persistent across sessions.

# Data Structures & Algorithm

## Methodology

The project was developed in C++ using an object-oriented approach. A single class SmartCityTrafficNavigationSystem encapsulates all data and functionality. The development followed these steps:

First, the graph representation was designed using both an adjacency list and a 2D weight matrix stored dynamically so the graph can grow at runtime.

Second, file handling was implemented to load intersection names and road distances from text files at startup, making the system persistent across sessions.

Third, core features were built one by one — display map, add intersection, add road, and shortest path using Dijkstra's Algorithm.

Finally, input validation was added throughout to handle invalid user inputs gracefully.

## Data Structures Used

- Adjacency List (using list<int>) — stores outgoing road connections for each intersection. Since the graph is directed, only one direction is stored per road.
- 2D Dynamic Array (matrix) — stores road distances between intersections. matrix[i][j] gives the distance of the one-way road from intersection i to intersection j. Allocated dynamically using new so it can grow when new intersections are added.
- String Array (routesName) — stores the name of each intersection. Also dynamically allocated so it grows with new additions.
- Boolean Array (vIntersection) — tracks which intersections have been visited during Dijkstra's execution.
- Integer Array (pIntersection) — stores the previous intersection on the shortest known path. Used to trace back the full route after Dijkstra completes.

## Algorithm Used — Dijkstra's Algorithm

Dijkstra's Algorithm finds the shortest path from a source intersection to all other intersections in a weighted directed graph.

**Step 1** — Initialize: Set distance of all intersections to infinity (99999). Set source distance to 0. Mark all as unvisited. Set all previous intersections to -1.

**Step 2** — Pick minimum: From all unvisited intersections, find the one with the smallest known distance using a simple loop.

**Step 3** — Relax neighbors: For each neighbor of the picked intersection, check if going through the current intersection gives a shorter distance. If yes, update its distance and record the current intersection in pIntersection.

**Step 4** — Repeat: Steps 2 and 3 repeat V times (once for each intersection).

**Step 5** — Trace path: Follow pIntersection array backwards from destination to source to reconstruct the full route.

## Time Complexity

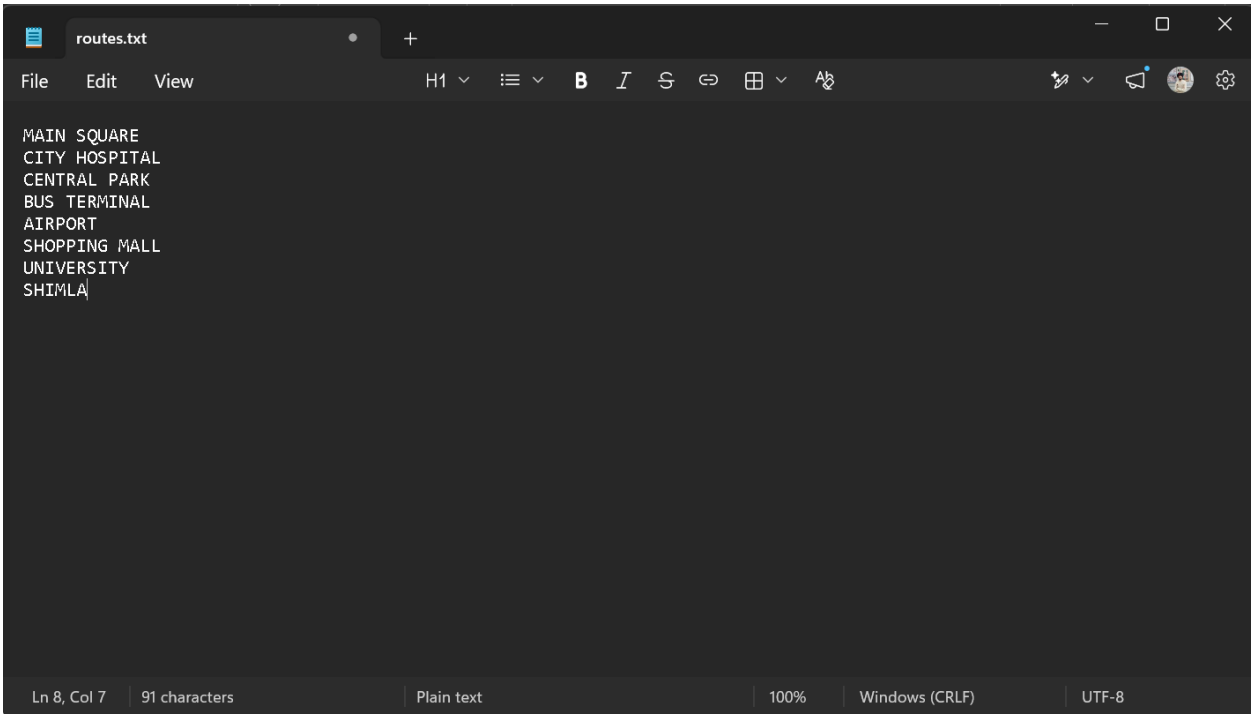
Shortest Path (Dijkstra with array): $O(V^2)$ — simple array used to find minimum distance node.
Add Intersection: $O(V^2)$ — due to matrix resize and copy.
Add Road: O(1) — direct matrix and list update.
Display Map: $O(V^2)$ — loops through matrix.

## Space Complexity

Adjacency List: O(V + E)
Distance Matrix: $O(V^2)$
Dijkstra arrays (cDistance, pIntersection, vIntersection): O(V)
Overall: $O(V^2)$ dominated by the distance matrix.

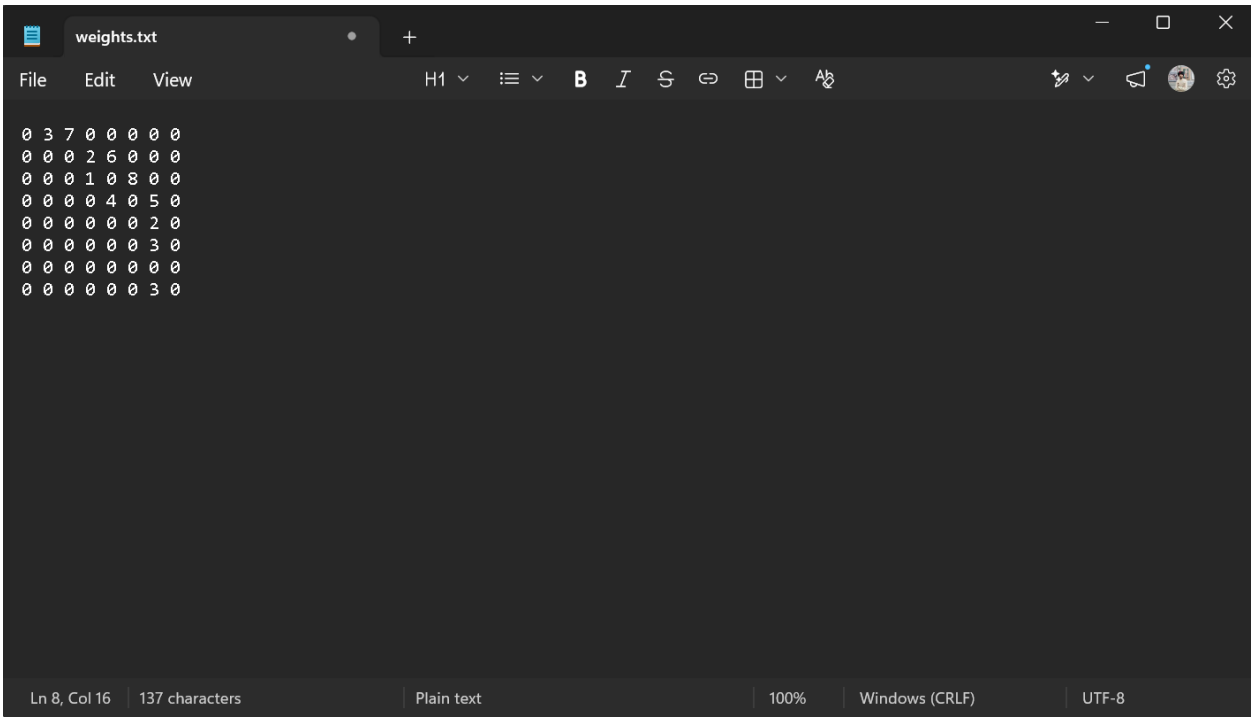
## File Handling

Two files are used:
**routes.txt** — stores intersection names, one per line. Read using getline at startup and written back on exit.



# Data Structures & Algorithm

**weights.txt** — stores the distance matrix row by row, numbers separated by spaces. Read using >> operator at startup and written back on exit.




When the user exits, both files are automatically updated to save any new intersections or roads added during the session. This means data persists across program runs.

## Feature 1 — Display City Map (Option 4)

**Explanation:** Prints all intersections and their outgoing roads with distances. If an intersection has no outgoing roads, it prints "Can't go to other routes from here."

**How the code works:** Outer loop runs for every intersection. Inner loop checks each column of the matrix row. If matrix[i][j] is not zero, it prints the destination name and distance. A counter tracks how many zeros are found — if all are zero, the no-route message is printed.

**Output:**

```
============================== Smart City Traffic Navigation System ==============================
========== Main Menu ==========
1) Add Intersection
2) Add Road
3) Find Shortest Route
4) Display City Map
5) Exit
Enter your option= 4

MAIN SQUARE -> CITY HOSPITAL (3 km) CENTRAL PARK (7 km)
CITY HOSPITAL -> BUS TERMINAL (2 km) AIRPORT (6 km)
CENTRAL PARK -> BUS TERMINAL (1 km) SHOPPING MALL (8 km)
BUS TERMINAL -> AIRPORT (4 km) UNIVERSITY (5 km)
AIRPORT -> UNIVERSITY (2 km)
SHOPPING MALL -> UNIVERSITY (3 km)
UNIVERSITY -> Can't go to other routes from here.
SHIMLA -> UNIVERSITY (3 km)
```

## Feature 2 — Add Intersection (Option 1)

**Explanation:** Allows the user to add a new intersection to the city map at runtime. The name is converted to uppercase for consistency and checked for duplicates before adding.

**How the code works:** The name is uppercased using toupper character by character. Then all existing names are checked — if a match is found, it returns early. Otherwise V is incremented, and routesName, matrix, and l are all resized by allocating new arrays of size V, copying old data, deleting old memory, and pointing to the new arrays.

**Output:**

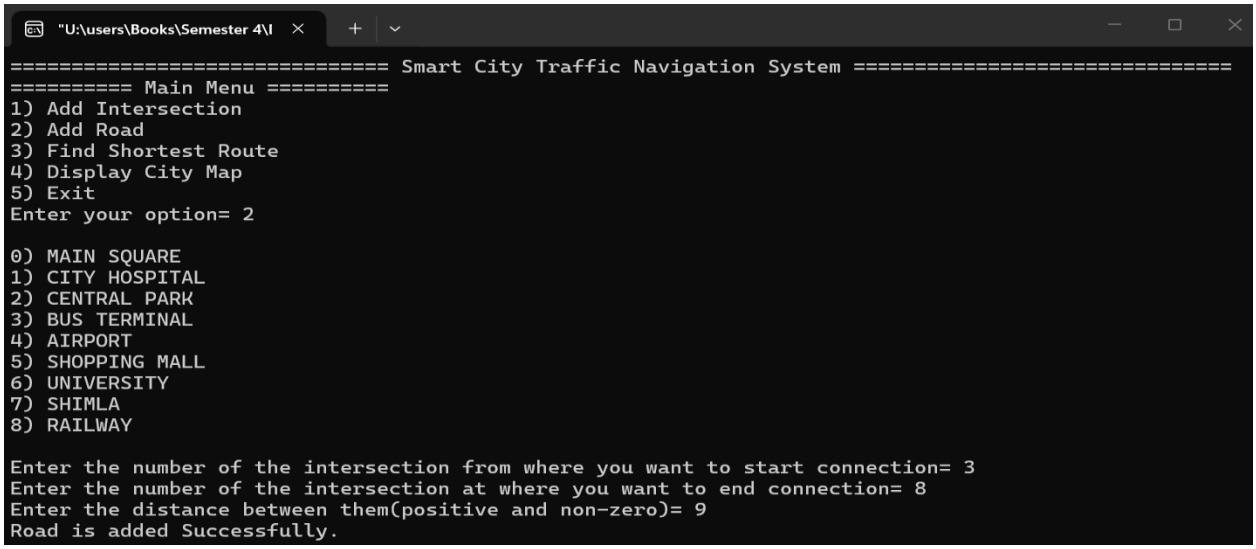
```
================================ Smart City Traffic Navigation System ================================
========== Main Menu ==========
1) Add Intersection
2) Add Road
3) Find Shortest Route
4) Display City Map
5) Exit
Enter your option= 1

Enter the Intersection Name= Railway
Intersection is added successfully.
```

## Feature 3 — Add Road (Option 2)

**Explanation:** Adds a one-way road between two intersections with a given distance. User selects source, destination, and distance. Input is validated — negative distance, equal intersections, and out-of-range numbers are all rejected.

**How the code works:** First checks matrix[u][v] — if not zero, road already exists. Otherwise sets matrix[u][v] = distance and adds v to l[u]. Only one direction is added since roads are one-way.

**Output:**



# Data Structures & Algorithm

## Feature 4 — Find Shortest Route (Option 3)

**Explanation:** Finds the shortest route between two intersections and prints the full path with total distance. Uses Dijkstra's Algorithm.

**Dry Run Example — Main Square (0) to University (6):**

Initial cDistance[]: {0, inf, inf, inf, inf, inf, inf}

Iteration 1 — Pick Main Square (dist=0): Relax City Hospital: 0+3=3, Relax Central Park: 0+7=7 cDistance[]: {0, 3, 7, inf, inf, inf, inf}

Iteration 2 — Pick City Hospital (dist=3): Relax Bus Terminal: 3+2=5, Relax Airport: 3+6=9 cDistance[]: {0, 3, 7, 5, 9, inf, inf}

Iteration 3 — Pick Bus Terminal (dist=5): Relax Airport: 5+4=9 → already 9, no update. Relax University: 5+5=10 cDistance[]: {0, 3, 7, 5, 9, inf, 10}

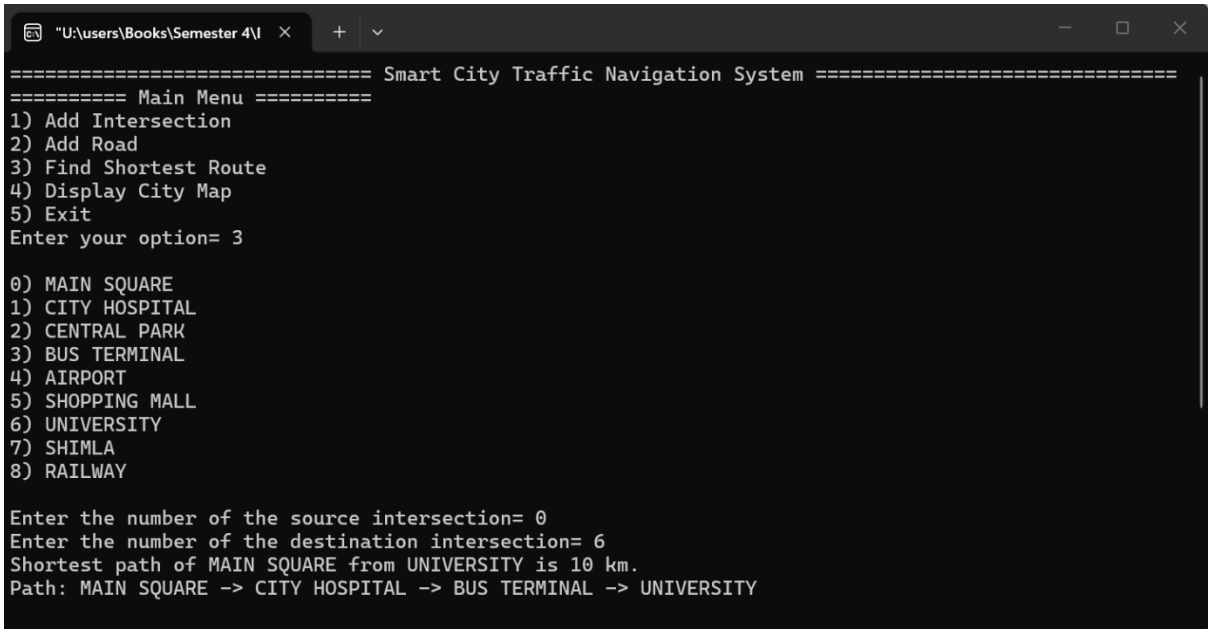
Iteration 4 — Pick Central Park (dist=7): Relax Bus Terminal: 7+1=8 → already 5, no update. Relax Shopping Mall: 7+8=15 cDistance[]: {0, 3, 7, 5, 9, 15, 10}

Iteration 5 — Pick Airport (dist=9): Relax University: 9+2=11 → already 10, no update

Iteration 6 — Pick University (dist=10): destination reached.

Final shortest distance: 10 km Path: Main Square → City Hospital → Bus Terminal → University

**Output:**




## Feature 5 — Invalid Option Handling

**Explanation:** If the user enters a number outside 1-5, the default case in the switch statement prints "Invalid Option" and the menu is shown again. For options 2 and 3, additional validation checks that intersection numbers are within range, not equal to each other, and that distance is positive and non-zero.

# Data Structures & Algorithm

**Output:**

```
============================== Smart City Traffic Navigation System ==============================
========== Main Menu ==========
1) Add Intersection
2) Add Road
3) Find Shortest Route
4) Display City Map
5) Exit
Enter your option= 7

Invalid Option
```

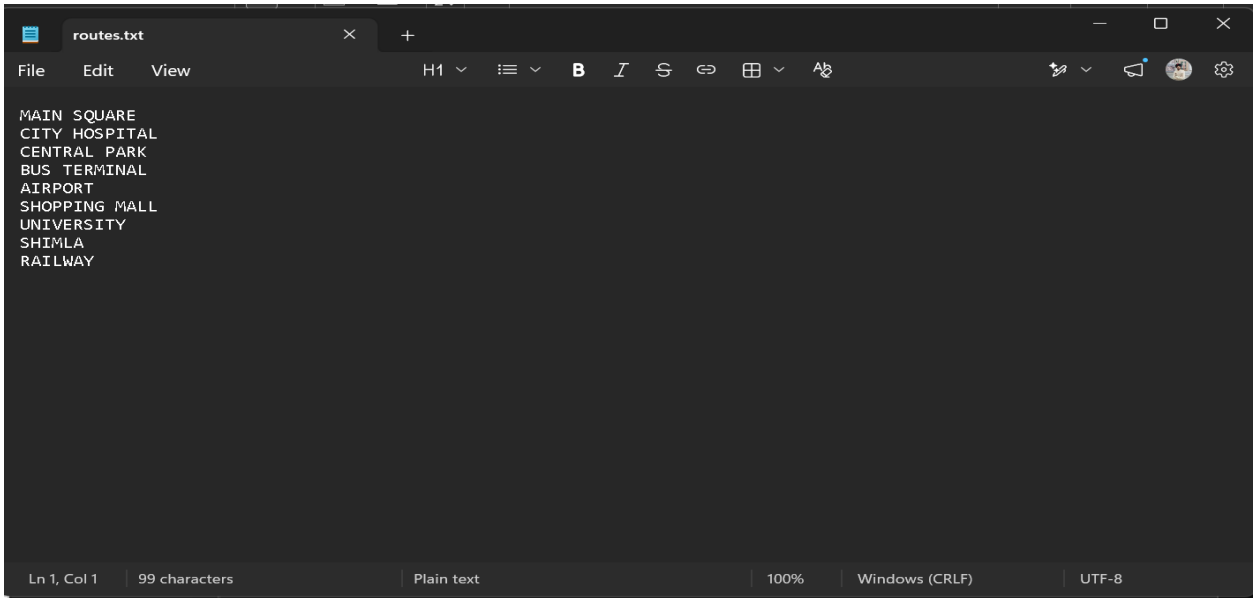
## Feature 6 — Exit (Option 5)

**Explanation:** When user selects 5, eS becomes true and the while loop ends. Before exiting, exiting() is called which saves all current data back to routes.txt and weights.txt. This means any intersections or roads added during the session are preserved for the next run.

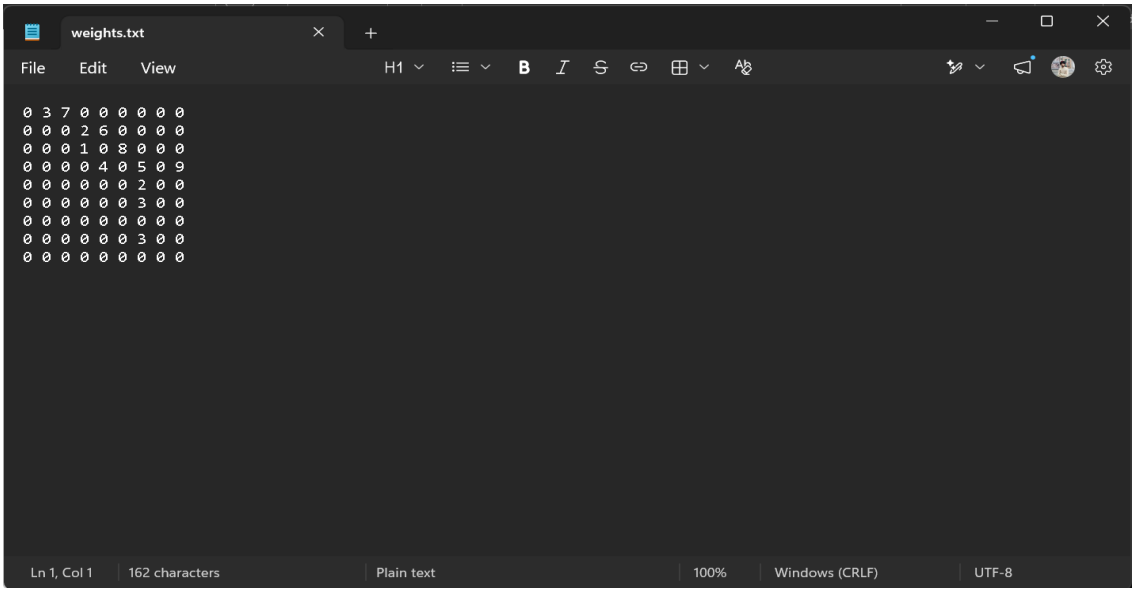
**Output:**

```
============================== Smart City Traffic Navigation System ==============================
========== Main Menu ==========
1) Add Intersection
2) Add Road
3) Find Shortest Route
4) Display City Map
5) Exit
Enter your option= 5

Thanks for using Smart City Traffic Navigation System. Come Back anytime!

Process returned 0 (0x0)   execution time : 155.825 s
Press any key to continue.
```

## Updated txt files:



# Data Structures & Algorithm




## Code:

```
#include<iostream>
#include<fstream>
#include<list>
#include<queue>

#define inf 99999
using namespace std;

/*
   Project No. 9: Smart City Traffic Navigation System using Dijkstra Algorithm
   Find shortest routes between city intersections in a weighted graph.
   Created by: 2024-CE-25 --> Asmad Soban
           2024-CE-46 --> Ubaid Ali
*/
/*
   Class Variable Full Names                | Details
   V = Vertex                               | Stores how many vertices a graph contains.
   l = list                                 | A list pointer. This will make list of the neighbor vertices of a vertex.
   matrix                                   | A 2D array pointer. This will store the weights of routes coming from txt file.
   routesName                               | A string array pointer which will store the names of routes.
   rG = routeGetter                         | This will hold the one route at a time coming from txt file.
   routes                                   | This is a queue which will store all the routes coming from txt file.
   nRoutesName = newRoutesName              | a helper of routesName so that we can easily enter new intersection.
```

# Data Structures & Algorithm

nMatrix = newMatrix | a helper of matrix so that we can easily update weights of intersections.
nL = newList | a helper of l so that we can easily update edges of intersections.
sDistance = shortestDistance | This is an array that will store the shortest distance from source to each route.
pDistance = previousDistance | This is an array that will store the previous distance from source to each route.
sPath = smallestPath | It will store the final smallest path.
visited | It will store the status of a vertex that it is relaxed or not.
vIntersection = visitedIntersection | Holds information of intersection that it is visited or not.
pIntersection = previous Intersection | stores the previous city on the best known path to each city.Starts at -1 (meaning "no predecessor known"). This is what lets us reconstruct the path at the end
minValue = minimumValue | this will store the minimum value of the current vertex which is in process of relaxation.
minVertex = minimumVertex | this will store the vertex of minimum value.

| Main Variable Full Names | Details |
|---|---|
| SCTNS | Object of class SmartCityTrafficNavigationSystem |
| mainMenu | This will contain the services list given to the user. |
| uC = userChoice | take user choice |
| eS = exitStatus | check if user wants to exit program or not |
| iN = intersectionName | Holds the new incoming intersection name. |
| src = source | This will be use to store the starting number of intersection. |
| destination | This will be use to store the ending number of intersection. |
| distance | This will store the weight of src and destination distance. |

| Functions Name | Details |
|---|---|
| addEdge | Will be use to add Edges to the vertices. |
| displayMap | This will display the map of the city. (Service 4) |
| displayRoutes | This will display all the routes(Intersections). |
| addRoad | This will make a new road connection between intersections.(Service 2) |
| addIntersection | This will make a new intersection.(Service 1) |
| exiting | This is a function that will run when we exit. This is made because we want to update our txt files when exit.(Service 5) |
| shortestPath | This will calculate the shortest path.(Service 3) |

```
*/

class SmartCityTrafficNavigationSystem{
   int V;
   list<int> *l;
   int **matrix;
   string *routesName;
//    int sPath;

public:
   SmartCityTrafficNavigationSystem(){
      string rG;
      queue<string> routes;

      ifstream routesInput("files/routes.txt");
      while(getline(routesInput, rG)){
         routes.push(rG);
      }
      routesInput.close();

      this->V = routes.size();
      l = new list<int> [V];
      matrix = new int*[V];
      routesName = new string[V];

      for(int i=0; i<V; i++){
         matrix[i] = new int[V]{0};
      }

      ifstream weightsInput("files/weights.txt");
      for(int i=0; i<V; i++){
         for(int j=0; j<V; j++){
            weightsInput>>matrix[i][j];
         }
      }
      weightsInput.close();

      for(int i=0; i<V; i++){
         routesName[i] = routes.front();
         routes.pop();
      }
```

```
    addEdges();
}

void addEdges(){
    for(int i=0; i<V; i++){
        for(int j=0; j<V; j++){
            if(matrix[i][j] != 0){
                l[i].push_back(j);
            }
        }
    }
    /* for(int i=0; i<V; i++){
        cout<<i<<"-> ";
        for(int j: l[i]){
            cout<<j<<" ";
        }
        cout<<endl;
    } */
}

void displayMap(){
    for(int i=0; i<V; i++){
        int counter = 0;
        cout<<routesName[i]<<" -> ";
        for(int j=0; j<V; j++){
            if(matrix[i][j] != 0) cout<<routesName[j]<<" ("<<matrix[i][j]<<" km) ";
            else counter++;
            if(counter == V) cout<<"Can't go to other routes from here.";
        }
        cout<<endl;
    }
}

void displayRoutes(){ for(int i=0; i<V; i++) cout<<i<<") "<<routesName[i]<<endl;}

void addIntersection(string name){
    for(int i=0; i<name.length(); i++)
        name[i] = toupper(name[i]);

    for(int i=0; i<V; i++){
        if(name == routesName[i]){
```

```
            cout<<"Already Exists"<<endl;
            return;
        }
    }

    this->V++;
    string *nRoutesName = new string[V];
    for(int i=0; i<V; i++){
        if(i == V-1) nRoutesName[i] = name;
        else nRoutesName[i] = routesName[i];
    }
    delete[] routesName;
    routesName = nRoutesName;


    int **nMatrix = new int*[V];
    for(int i=0; i<V; i++){
        nMatrix[i] = new int[V]{0};
    }
    for(int i=0; i<V-1; i++){
        for(int j=0; j<V-1; j++){
            nMatrix[i][j] = matrix[i][j];
        }
    }
    for(int i=0; i<V-1; i++){ delete[] matrix[i];}
    delete[] matrix;
    matrix = nMatrix;

    list<int> *nL = new list<int> [V];
    for(int i=0; i<V-1; i++){
        nL[i] = l[i];
    }
    delete[] l;
    l = nL;

    cout<<"Intersection is added successfully."<<endl;
}




void addRoad(int u, int v, int d){
```

```
    if(matrix[u][v] != 0){
        cout<<"Road already exists between them."<<endl;
        return;
    }
    matrix[u][v] = d;
    l[u].push_back(v);
    cout<<"Road is added Successfully."<<endl;
}


void exiting(){
    ofstream routesOutput("files/routes.txt");
    for(int i=0; i<V; i++){
        if(i == V-1) routesOutput<<routesName[i];
        else routesOutput<<routesName[i]<<"\n";
    }
    routesOutput.close();

    ofstream matrixOutput("files/weights.txt");
    for(int i=0; i<V; i++){
        for(int j=0; j<V; j++){
            matrixOutput<<matrix[i][j];
            if(j != V-1) matrixOutput<<" ";
        }
        matrixOutput<<"\n";
    }
    matrixOutput.close();
    cout<<"Thanks for using Smart City Traffic Navigation System. Come Back anytime!";
}

void shortestPath(int u, int v){
    int sDistance[V], pDistance[V];
    bool visited[V];

    for(int i=0; i<V; i++){
        sDistance[i] = inf;
        pDistance[i] = -1;
        visited[i] = false;
    }
    sDistance[u] = 0;

    for(int i=0; i<V; i++){
```

```
        int minValue = inf, minVertex = -1;
        for(int j=0; j<V; j++){
          if(sDistance[j]<minValue && !visited[j]){
            minValue = sDistance[j];
            minVertex = j;
          }
        }
        if(minVertex == -1) break;
        visited[minVertex] = true;

        for(int j: l[minVertex]){
          if(matrix[minVertex][j] != 0 && !visited[j]){
            if(sDistance[minVertex]+matrix[minVertex][j]<sDistance[j]){
              sDistance[j] = sDistance[minVertex]+matrix[minVertex][j];
              pDistance[j] = minVertex;
            }
          }
        }

      }
      if(sDistance[v] == inf) cout<<"Path not exist."<<endl;
      else{
        cout<<"Shortest path of "<<routesName[u]<<" from "<<routesName[v]<<" is
"<<sDistance[v]<<" km."<<endl;
        list<int> path;
        for(int i=v; i!=-1; i=pDistance[i]){
          path.push_front(i);
        }
        cout<<"Path: ";
        for(int i: path){
          cout<<routesName[i];
          if(i != v) cout<<" -> ";
        }
        cout<<endl;
      }


    }

    int getV(){return V;}
```

```
};



int main(){
  string mainMenu[] = {"Add Intersection", "Add Road", "Find Shortest Route", "Display City Map", "Exit"};
  int uC = 0;
  bool eS = false;

  SmartCityTrafficNavigationSystem SCTNS;
  while(!eS){
    cout<<"================================ Smart City Traffic Navigation System ================================"<<endl;
    cout<<"========== Main Menu ==========="<<endl;
    for(int i=0; i<5; i++){
      cout<<i+1<<") "<<mainMenu[i]<<endl;
    }
    cout<<"Enter your option= ";
    cin>>uC;
    cout<<endl;
    switch(uC){
      case 1:{
        string iN;
        // SCTNS.displayRoutes();
        cout<<"Enter the Intersection Name= ";
        cin.ignore();
        getline(cin, iN);
        SCTNS.addIntersection(iN);
        break;
      }case 2:{
        int src, destination, distance;
        SCTNS.displayRoutes();
        cout<<endl;
        cout<<"Enter the number of the intersection from where you want to start connection= ";
        cin>>src;
        if(src<0 or src>SCTNS.getV()){
          cout<<"Starting intersection is invalid"<<endl;
          break;
        }
        cout<<"Enter the number of the intersection at where you want to end connection= ";
```

```
    cin>>destination;
    if(destination<0 or destination>SCTNS.getV()){
      cout<<"Ending intersection is invalid"<<endl;
      break;
    }
    if(src == destination) {
      cout<<"Entered values are equal."<<endl;
      break;
    }
    cout<<"Enter the distance between them(positive and non-zero)= ";
    cin>>distance;
    if(distance>0) SCTNS.addRoad(src, destination, distance);
    else cout<<"Entered distance is not valid."<<endl;
    break;
  }case 3:{
    int src, destination;
    SCTNS.displayRoutes();
    cout<<endl;
    cout<<"Enter the number of the source intersection= ";
    cin>>src;
    if(src<0 or src>SCTNS.getV()){
      cout<<"Source intersection is invalid"<<endl;
      break;
    }
    cout<<"Enter the number of the destination intersection= ";
    cin>>destination;
    if(destination<0 or destination>SCTNS.getV()){
      cout<<"Destination intersection is invalid"<<endl;
      break;
    }
    if(src != destination) SCTNS.shortestPath(src, destination);
    else cout<<"Entered values are equal."<<endl;
    break;
  }case 4:
    SCTNS.displayMap();
    break;
  case 5:
    eS = true;
    SCTNS.exiting();
    break;
  default:
    cout<<"Invalid Option"<<endl;
```

```
            break;
        }
        cout<<endl;
        cout<<endl;
    }


    return 0;
}
```

## Real-World Application

This system simulates how GPS navigation works in real city traffic. Dijkstra's Algorithm is the core of Google Maps, Waze, and similar applications. The directed graph models one-way streets accurately. Dynamic addition of intersections and roads simulates how city infrastructure expands over time. File handling ensures the city map is persistent, just like a real database-backed navigation system.

## Conclusion

This project successfully implements a Smart City Traffic Navigation System using a directed weighted graph and Dijkstra's Algorithm. Users can dynamically add intersections and roads, find shortest routes with full path tracing, and view the complete city map. File handling ensures data is saved and loaded automatically. Input validation is included throughout. The system demonstrates how core DSA concepts like graphs, adjacency lists, and shortest path algorithms apply directly to real-world navigation problems.